Documento de apoyo (Sesión 4, Unidad 2)

Herramientas ETL y tendencias modernas

Tipos de herramientas, criterios de selección y casos de uso

Javier Andrés Valdez González / Auxiliar

Seminario de Sistemas 2 — Sección P

Universidad de San Carlos de Guatemala

Febrero de 2026

Resumen

Este documento complementa el material previo sobre fundamentos y operación del proceso ETL, enfocándose en el *panorama de herramientas* disponibles hoy y en las *tendencias modernas* de integración de datos. Se proponen tipologías prácticas (ETL visual, *data replication* tipo ELT, orquestación, *dataflow* y CDC/streaming, y capas de transformación), se presentan ejemplos representativos comerciales y *open source*, se describen servicios ETL administrados en la nube y se establecen criterios técnicos para seleccionar herramientas según volumen, latencia, gobernanza y mantenibilidad. Finalmente, se discuten tendencias actuales (modern data stack, lakehouse, DataOps/observabilidad y tiempo casi real) mediante casos de uso reales orientados a analítica y OLAP.

Palabras clave: integración de datos, ETL, ELT, orquestación, CDC, Airflow, dbt, Airbyte, ADF, AWS Glue, Data Fusion.

Índice

1. Alcance y premisas **4**

2. Tipos de herramientas ETL (y herramientas cercanas a ETL) **5**
2.1. Plataformas ETL visuales (*low-code/no-code*) . . . 5
2.2. Herramientas de *data replication* (ELT orientado a conectores) . . . 5
2.3. Capas de transformación en el destino (dbt y similares) . . . 6
2.4. Orquestación y automatización de workflows (no es ETL, pero lo sostiene) . . . 6
2.5. Automatización de flujos de datos (*dataflow*) . . . 6
2.6. CDC y streaming para latencias bajas . . . 7

3. Ejemplos de herramientas comerciales y Open Source **8**
3.1. Mapa rápido por categoría . . . 8
3.2. Notas de evaluación para herramientas open source . . . 8
3.3. Notas de evaluación para herramientas comerciales . . . 9

4. Herramientas ETL en la nube **10**
4.1. Servicios administrados representativos . . . 10
4.2. Patrones comunes en la nube . . . 10
4.3. Costos y *trade-offs* . . . 10
4.4. Integración casi real y cargas continuas . . . 11

5. Criterios de selección **12**
5.1. Criterios técnicos (checklist) . . . 12
5.2. Matriz de puntuación (plantilla) . . . 12
5.3. Regla práctica para escenarios OLAP . . . 13

6. Tendencias actuales **14**
6.1. Del ETL monolítico al *modern data stack* . . . 14
6.2. Lakehouse y pipelines declarativos . . . 14
6.3. CDC/streaming como estándar para latencias bajas . . . 14
6.4. Observabilidad y gobernanza “por defecto” . . . 14

7. Casos de uso reales **16**
7.1. Caso 1: Analítica de ventas (batch nocturno) para cubo OLAP . . . 16
7.2. Caso 2: Marketing y analítica SaaS (ELT con conectores) . . . 16

7.3. Caso 3: Inventario de alta rotación (near real-time con CDC) 16
7.4. Caso 4: Integración de archivos y múltiples destinos (dataflow) 17
7.5. Caso 5: Carga continua serverless en warehouse 17

8. Conclusiones **18**

1. Alcance y premisas

Este documento asume que el lector ya conoce (i) el propósito de ETL/ELT en arquitecturas analíticas, (ii) la idea de *staging*, cargas incrementales y control de calidad, y (iii) el rol del modelado dimensional para OLAP. El objetivo aquí no es re-explicar el proceso, sino ampliar el **criterio de ingeniería** para responder preguntas típicas en entornos reales:

- *¿Qué tipo de herramienta necesito para mi escenario (batch vs. casi real, on-prem vs. nube, low-code vs. código)?*
- *¿Cómo comparo alternativas comerciales y open source sin caer en "listas de nombres"?*
- *¿Qué tendencias están cambiando la forma de construir pipelines y qué riesgos introducen?*

A lo largo del documento se usa **OLAP** como contexto de consumo (modelo estrella, *data marts*, cubos/tablas) y, por tanto, se enfatizan propiedades como: consistencia semántica, trazabilidad, repetibilidad, y capacidad de operar cargas incrementales.

2. Tipos de herramientas ETL (y herramientas cercanas a ETL)

En la práctica, *"herramienta ETL"* se usa como etiqueta para tecnologías diferentes que cumplen partes del ciclo de integración (extraer, mover, transformar, orquestar, observar, versionar). Una clasificación útil para evaluar soluciones (sin depender de marcas) es la siguiente.

2.1. Plataformas ETL visuales (*low-code/no-code*)

Son entornos con diseño de flujos por componentes (fuente–transformación–destino), conectores preconstruidos y ejecución administrada (o desplegable). Ventajas comunes:

- **Productividad** para pipelines estándar (mover, limpiar, mapear).
- **Monitoreo integrado** (*runs*, estados, reintentos, alertas).
- **Gobernanza** más directa (roles, auditoría, catálogos, entornos).

Riesgos típicos: dependencia de componentes propietarios, dificultad para versionar flujos complejos como código, y coste por ejecución/tiempo (en ofertas SaaS). En la nube, servicios como Azure Data Factory se posicionan explícitamente como *data integration/ETL* con UI y orquestación [13].

2.2. Herramientas de *data replication* (ELT orientado a conectores)

Estas herramientas priorizan **extraer y cargar** a un destino analítico (warehouse/lakehouse) con mínima transformación previa. La transformación se desplaza hacia capas posteriores (SQL, dbt, notebooks), siguiendo el patrón ELT.

Ejemplos representativos: Fivetran (gestionado) [11] y Airbyte (open source/gestionado) [1], [2]. Este enfoque es especialmente atractivo cuando:

- la mayoría de fuentes son SaaS/APIs y se necesita sincronización frecuente;
- el equipo prefiere transformar "cerca del warehouse" con SQL y pruebas;
- se requiere escalar conectores rápidamente sin diseñar flujos complejos.

2.3. Capas de transformación en el destino (dbt y similares)

En el *modern data stack*, la capa de transformación se formaliza como **proyecto versionado** que genera modelos tabulares, documentación y pruebas automatizadas. dbt Core se presenta como herramienta open source para transformar datos *dentro* del destino [9] y genera documentación del proyecto como artefacto reproducible [8]. En escenarios OLAP, esta capa se usa para:

- estandarizar dimensiones conformadas y jerarquías;
- definir hechos agregables (sumarizables) por grano;
- mantener un diccionario de métricas y lógica de negocio versionada.

2.4. Orquestación y automatización de workflows (no es ETL, pero lo sostiene)

Un pipeline robusto raramente es un solo "job". Suele incluir dependencias (staging → dims → facts → validaciones → refresco de cubo). Apache Airflow se define como plataforma open source para **desarrollar, calendarizar y monitorear workflows batch** [6]. Un orquestador es clave para:

- controlar dependencias y reintentos con idempotencia;
- estandarizar parametrización por ambiente (dev/qa/prod);
- instrumentar alertas y auditoría de ejecución por DAG.

2.5. Automatización de flujos de datos (*dataflow*)

Herramientas de *dataflow* se enfocan en mover y enrutar datos entre sistemas (integración, *routing*, *backpressure*). Apache NiFi describe su propósito como automatizar el flujo de datos entre sistemas [5]. Es útil cuando hay:

- múltiples fuentes y destinos heterogéneos;
- necesidad de enrutar por reglas (p. ej., cuarentena vs. carga);
- ingestión continua de archivos, colas o eventos con control de presión.

2.6. CDC y streaming para latencias bajas

Cuando se busca latencia casi real, el "ETL" se aproxima a integración por eventos. Debezium se presenta como plataforma distribuida open source para **captura de cambios (CDC)** [10]. Kafka Connect ofrece modos de ejecución y operación de conectores para import/export de datos en flujos [4]. En analítica, CDC/streaming permite:

- alimentar tablas incrementales con granularidad de cambio;
- habilitar dashboards de operación (minutos/segundos);
- reducir ventanas batch, a costa de mayor complejidad operativa.

3. Ejemplos de herramientas comerciales y Open Source

Esta sección no busca *"la lista definitiva"*. El objetivo es mostrar cómo mapear herramientas a la tipología anterior, y qué **preguntas correctas** hacer al evaluarlas.

3.1. Mapa rápido por categoría

Categoría	**¿Qué resuelve mejor?**	**Ejemplos (referenciales)**
ETL visual (low-code)	Pipelines estándar, monitoreo integrado, conectores + orquestación	Azure Data Factory [13]; Cloud Data Fusion [12]
ELT / replication	Cargar rápido muchas fuentes (SaaS/APIs/BD) al warehouse/lake	Fivetran [11]; Airbyte [1]
Transformación en destino	Modelado, pruebas, documentación y linaje lógico sobre SQL	dbt Core [9]
Orquestación	Dependencias, calendarización, reintentos, observabilidad de jobs	Apache Airflow [6]
Dataflow	Enrutamiento, ingestión continua, backpressure, cuarentenas	Apache NiFi [5]
CDC/Streaming	Cambios incrementales, near real-time, integración por eventos	Debezium [10]; Kafka Connect [4]

3.2. Notas de evaluación para herramientas open source

Las herramientas open source suelen ser atractivas por costo y extensibilidad, pero su "costo total" depende de:

O1. **Operación**: instalación, upgrades, HA, observabilidad, backups.

O2. **Conectores reales**: no es solo "existe el conector", sino si soporta *incremental*, *schema evolution*, límites API, y autenticación moderna.

O3. **Gobernanza**: roles, auditoría, encriptación, secretos, segregación de ambientes.

O4. Ciclo de desarrollo: CI/CD, revisión, pruebas, versionado como código.

Por ejemplo, Airbyte enfatiza un catálogo amplio y una plataforma de integración open source [1], [2]; esto acelera prototipos, pero el equipo debe definir prácticas de operación y control de cambios para sostener entornos productivos.

3.3. Notas de evaluación para herramientas comerciales

Las plataformas comerciales tienden a ofrecer:

- SLAs, soporte, conectores mantenidos y cumplimiento empresarial;
- integraciones con IAM corporativo y auditoría;
- UI, monitoreo y *governance* integrados.

Sin embargo, conviene revisar dependencias de licenciamiento, costos por ejecución (en nube), límites por conector, y portabilidad. Como ejemplo histórico, Talend Open Studio fue discontinuado a inicios de 2024 [16]; y Qlik mantiene páginas de ciclo de vida/fin de soporte para productos Talend [14]. Este tipo de eventos es relevante al evaluar riesgo de continuidad y estrategia de migración.

4. Herramientas ETL en la nube

La nube estandarizó una clase de herramientas: **servicios administrados** para integración, orquestación y transformación. Su ventaja principal no es "hacer ETL", sino reducir carga operativa: escalado, infraestructura, integración con identidades y monitoreo.

4.1. Servicios administrados representativos

- **Azure Data Factory (ADF):** se describe como servicio ETL e integración de datos para crear workflows que mueven y transforman datos a escala [13].
- **AWS Glue:** se describe como servicio *serverless* de integración de datos para descubrir, preparar, mover e integrar datos desde múltiples fuentes [3].
- **Cloud Data Fusion:** se describe como servicio administrado, *cloud-native*, para construir y gestionar pipelines [12].

4.2. Patrones comunes en la nube

C1. Ingestión hacia un lake/warehouse: carga por lotes o micro-batches a almacenamiento/warehouse.

C2. Transformación con motores administrados: Spark/SQL administrado o jobs de transformación.

C3. Orquestación central: pipelines con dependencias, *retry*, alertas y auditoría.

C4. Seguridad integrada: IAM, secretos, redes privadas, *private endpoints*.

4.3. Costos y *trade-offs*

En la nube, el costo está ligado a consumo (tiempo de ejecución, workers, throughput, conectores). Buenas prácticas al diseñar pipelines:

- preferir incrementales (CDC/marcas de agua) para no "pagar por volver a copiar todo";
- particionar por tiempo o claves de negocio para cargas y consultas;
- separar *dev/qa/prod* para evitar ejecuciones accidentales costosas.

4.4. Integración casi real y cargas continuas

Algunos warehouses ofrecen mecanismos **serverless** de carga continua. Por ejemplo, Snowpipe habilita cargar archivos en micro-batches "tan pronto como estén disponibles" [15]. En lakehouse/streaming, guías de Databricks describen pipelines ETL con CDC (incluyendo entrada desde Debezium) [7]. Estos enfoques reducen latencia, pero implican disciplina en:

- control de duplicación (exactly-once vs. at-least-once);
- manejo de cambios de esquema;
- control de calidad en flujos continuos.

5. Criterios de selección

Seleccionar una herramienta no es "elegir marca"; es alinear capacidades con requisitos. A continuación se propone un conjunto de criterios con enfoque académico-práctico para evaluar alternativas en proyectos de analítica/OLAP.

5.1. Criterios técnicos (checklist)

S1. Conectividad real: fuentes/destinos requeridos, autenticación, y soporte de incrementales.

S2. Volumen y rendimiento: throughput, paralelismo, particionamiento, y límites por conector.

S3. Latencia objetivo: batch (diario/hora) vs. micro-batch (minutos) vs. streaming (segundos).

S4. Transformaciones: complejidad (joins, SCD, enriquecimiento), expresividad (SQL/Python), y pruebas.

S5. Gobernanza y seguridad: IAM, auditoría, secretos, encriptación, redes privadas.

S6. Observabilidad: logs, métricas, *lineage* operativo, alertas y reintentos.

S7. Mantenibilidad: versionado, CI/CD, revisión por pares, ambientes, portabilidad.

S8. Costo total: licencias + operación + infraestructura + tiempo humano.

5.2. Matriz de puntuación (plantilla)

Una forma útil de formalizar la decisión es asignar pesos (importancia) y puntajes (capacidad). La Tabla 2 sirve como plantilla para clase/laboratorio.

Cuadro 2: Matriz de selección (plantilla).

Criterio	**Peso (1–5)**	**Puntaje (1–5)**	**Evidencia (¿cómo lo verifico?)**
Conectores e incrementales			Demo con una fuente real + corrida incremental
Observabilidad y reintentos			Logs por etapa + alertas + re-run idempotente

Transformación y pruebas	Caso con SCD/validaciones + suite de tests
Seguridad/IAM	Integración con roles + secretos + red privada
Costo total	Estimación de consumo/licencias + operación mensual
Portabilidad / lock-in	Exportación de pipeline / ejecución en otro entorno

5.3. Regla práctica para escenarios OLAP

Para analítica y cubos, suele funcionar bien la combinación:

- **Replicación/ingestión** (Airbyte/Fivetran o servicio cloud) →
- **Transformación versionada** (dbt/SQL) →
- **Orquestación** (Airflow/ADF) →
- **Publicación** (modelos estrella + materializaciones/particiones).

La decisión final depende de latencia, volumen y gobernanza: si se exige near real-time, CDC/streaming (Debezium/Kafka) pasa a ser componente central.

6. Tendencias actuales

La "modernidad" en ETL no es un conjunto fijo de herramientas, sino un cambio de énfasis: más automatización operativa, más transformación en el destino, y más disciplina de ingeniería (*DataOps*).

6.1. Del ETL monolítico al *modern data stack*

Se observa una separación de responsabilidades:

- ingestión/replicación (EL) como servicio o plataforma de conectores;
- transformación (T) dentro del warehouse/lakehouse con proyectos versionados (dbt) [9];
- orquestación y monitoreo como capa transversal [6].

Esto facilita modularidad: cada capa puede evolucionar sin rehacer todo el pipeline.

6.2. Lakehouse y pipelines declarativos

En entornos lakehouse, los pipelines combinan batch y streaming bajo un mismo sistema de almacenamiento/tabla. Documentación de Databricks presenta pipelines ETL con CDC, incluyendo integración con Debezium [7]. La tendencia relevante es **declarar** expectativas de calidad y dependencias, y dejar que el sistema ejecute/optimice.

6.3. CDC/streaming como estándar para latencias bajas

Debezium (CDC) [10] y Kafka Connect [4] se usan para mover cambios de bases operacionales hacia capas analíticas casi en tiempo real. Esto habilita:

- KPIs operativos (por hora/minuto);
- detección de anomalías/fraude con menor demora;
- sincronización de catálogos e inventarios en alta rotación.

El costo es complejidad: manejo de orden, duplicación, esquemas, y consistencia eventual.

6.4. Observabilidad y gobernanza "por defecto"

La tendencia DataOps impulsa:

- **pruebas** y documentación automatizadas en transformación [8];

- **monitoreo** y trazabilidad de ejecuciones en orquestación [6];
- prácticas de **CI/CD** para pipelines, evitando cambios manuales "en producción".

7. Casos de uso reales

Los siguientes casos están pensados para que el estudiante identifique requisitos y derive una arquitectura razonable, sin casarse con una sola marca.

7.1. Caso 1: Analítica de ventas (batch nocturno) para cubo OLAP

Contexto. Punto de venta (POS) y e-commerce generan transacciones; se requiere un cubo/tabla agregada por tiempo, tienda, categoría.

Requisitos. Latencia diaria; volumen medio; calidad alta; auditoría (cortes contables).

Arquitectura típica.

- ETL visual en la nube (ADF/Glue/Data Fusion) para extraer y cargar [3], [12], [13].
- Transformación en destino (SQL/dbt) para conformar dimensiones y construir hechos [9].
- Orquestación con dependencias y reconciliación (Airflow o el propio servicio) [6].

Riesgo clave. Correcciones retroactivas (devoluciones) → diseñar upserts/particiones y control de re-procesos.

7.2. Caso 2: Marketing y analítica SaaS (ELT con conectores)

Contexto. Datos dispersos en CRM, Ads, soporte, web analytics. Se necesita un modelo unificado para BI.

Requisitos. Muchas fuentes API; cambios frecuentes de campos; el equipo prefiere "cargar todo" y transformar después.

Arquitectura típica.

- Replicación al warehouse con conectores (Fivetran/Airbyte) [1], [11].
- Transformación con dbt para estandarizar métricas y documentación [8], [9].
- Orquestación para refrescar modelos y publicar tablas de consumo (Airflow) [6].

Riesgo clave. *Schema drift* en APIs → contratos mínimos, pruebas y alertas (DataOps).

7.3. Caso 3: Inventario de alta rotación (near real-time con CDC)

Contexto. Inventario cambia constantemente; operaciones necesitan alertas y tableros casi en tiempo real.

Requisitos. Latencia de minutos; consistencia razonable; tolerancia a consistencia eventual.

Arquitectura típica.

- CDC desde la BD operacional con Debezium [10].
- Transporte con Kafka Connect y topics (o conectores equivalentes) [4].
- Consumo en lakehouse/warehouse con micro-batches o streaming; guías de pipelines CDC (p. ej., Databricks) [7].

Riesgo clave. Duplicación/orden de eventos → claves idempotentes, ventanas, *watermarks*.

7.4. Caso 4: Integración de archivos y múltiples destinos (dataflow)

Contexto. Llegan archivos (CSV/JSON) desde proveedores; algunos registros son válidos, otros deben ir a cuarentena; se publican extractos a varios sistemas.

Requisitos. Enrutamiento por reglas, control de presión (*backpressure*), trazabilidad por archivo.

Arquitectura típica.

- Dataflow con NiFi para ingestión, validación y ruteo [5].
- Carga a staging y posteriormente transformaciones para el modelo analítico.

Riesgo clave. Falta de versionado de flujos → tratar la configuración como artefacto desplegable y auditable.

7.5. Caso 5: Carga continua serverless en warehouse

Contexto. Se reciben archivos en almacenamiento cloud y se desean visibles en el warehouse "en minutos".

Requisitos. Latencia corta sin administrar servidores; costos controlados.

Arquitectura típica. Snowpipe habilita carga en micro-batches cuando los archivos están disponibles [15]. Se complementa con transformación posterior y monitoreo de fallos.

Riesgo clave. Explosión de pequeños archivos → políticas de compactación y particionado.

8. Conclusiones

Las herramientas ETL modernas se entienden mejor como un **ecosistema** de componentes: ingestión, transformación, orquestación, dataflow y CDC/streaming. La selección adecuada depende de requisitos medibles (latencia, volumen, gobernanza, mantenibilidad) y de un diseño que preserve consistencia semántica para OLAP. En la práctica, combinar replicación/ingestión (Airbyte/Fivetran o servicios cloud) [1], [3], [11], [12], [13] con transformación versionada (dbt) [9] y orquestación (Airflow) [6] produce pipelines más modularizables y auditables. Para escenarios near real-time, CDC y streaming (Debezium/Kafka Connect) [4], [10] habilitan nuevos casos de uso, a cambio de mayor complejidad operativa.

Referencias

[1] Airbyte. «Airbyte documentation,» visitado 16 de feb. de 2026. dirección: https://docs.airbyte.com/

[2] airbytehq. «Airbyte (GitHub): The leading data integration platform for ETL/ELT,» visitado 16 de feb. de 2026. dirección: https://github.com/airbytehq/airbyte

[3] Amazon Web Services. «What is AWS Glue?» Visitado 16 de feb. de 2026. dirección: https://docs.aws.amazon.com/glue/latest/dg/what-is-glue.html

[4] Apache Software Foundation. «Kafka Connect User Guide,» visitado 16 de feb. de 2026. dirección: https://kafka.apache.org/41/kafka-connect/user-guide/

[5] Apache Software Foundation. «Apache NiFi Overview,» visitado 16 de feb. de 2026. dirección: https://nifi.apache.org/docs/nifi-docs/html/overview.html

[6] Apache Software Foundation. «What is Airflow? (Apache Airflow Documentation),» visitado 16 de feb. de 2026. dirección: https://airflow.apache.org/docs/apache-airflow/stable/index.html

[7] Databricks. «Tutorial: Build an ETL pipeline using change data capture,» visitado 16 de feb. de 2026. dirección: https://docs.databricks.com/aws/en/ldp/tutorial-pipelines

[8] dbt Labs. «About documentation (dbt Developer Hub),» visitado 16 de feb. de 2026. dirección: https://docs.getdbt.com/docs/build/documentation

[9] dbt Labs. «What is dbt? (dbt Developer Hub),» visitado 16 de feb. de 2026. dirección: https://docs.getdbt.com/docs/introduction

[10] Debezium. «Debezium: Distributed platform for Change Data Capture,» visitado 16 de feb. de 2026. dirección: https://debezium.io/

[11] Fivetran. «Getting started (Documentation),» visitado 16 de feb. de 2026. dirección: https://fivetran.com/docs/getting-started

[12] Google Cloud. «Cloud Data Fusion overview,» visitado 16 de feb. de 2026. dirección: https://docs.cloud.google.com/data-fusion/docs/concepts/overview

[13] Microsoft. «Introduction to Azure Data Factory,» visitado 16 de feb. de 2026. dirección: https://learn.microsoft.com/en-us/azure/data-factory/introduction

[14] Qlik. «Talend products lifecycle (Customer Support Statements),» visitado 16 de feb. de 2026. dirección: https://help.qlik.com/talend/en-US/customer-support-statements/Cloud/product-end-of-life-planning

[15] Snowflake. «Snowpipe,» visitado 16 de feb. de 2026. dirección: https://docs.snowflake.com/en/user-guide/data-load-snowpipe-intro

[16] Talend. «Update on the future of Talend Open Studio,» visitado 16 de feb. de 2026. dirección: https://www.talend.com/blog/update-on-the-future-of-talend-open-studio/